



Python is a **general-purpose**, dynamic, high-level and interpreted **programming language**.

Simple to read Easy to learn and open source

Known for its simplicity, readability, and versatility. It was created by **Guido van Rossum** and first released in **1991**.

Key Features of Python:

1. **Easy to Learn and Use:** Its simple syntax, similar to plain English, makes it an excellent language for beginners.
2. **Interpreted Language:** Python code is executed line by line, making it easier to debug.
3. **Dynamic Typing:** You don't need to declare variable types explicitly; Python figures it out during execution.
4. **Extensive Libraries:** Python has a rich ecosystem of libraries and frameworks (e.g., NumPy, Pandas, Django, Flask) that make development faster and more efficient.
5. **Object-Oriented:** Python supports object-oriented programming (OOP) principles, allowing the creation of classes, objects, inheritance, etc.
6. **Cross-Platform:** Python is available on almost every platform, making it easy to run Python programs on different operating systems like Windows, macOS, and Linux.
7. **Versatile:** Python is used in various fields, such as web development, data analysis, machine learning, artificial intelligence, automation, scientific computing, and more.

Common Use Cases:

- **Web Development:** Frameworks like Django and Flask are popular for building websites and web applications.
- **Data Science:** Libraries like NumPy, Pandas, and Matplotlib are widely used for data manipulation and visualization.
- **Machine Learning & AI:** Libraries like TensorFlow, Keras, and scikit-learn make Python a go-to language for machine learning and artificial intelligence projects.
- **Scripting and Automation:** Python can automate tasks, interact with APIs, and work with files and directories.
- **Game Development:** Libraries like Pygame allow developers to create games using Python.

Download, and Install

Search download python – Download python – open file – install now and close

[www.python.org > downloads](https://www.python.org/downloads/)

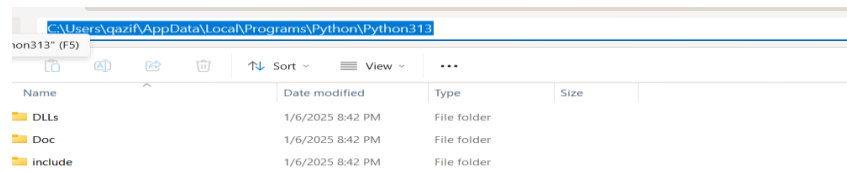
[Download Python | Python.org](https://www.python.org/downloads/)

Find the latest version of **Python** for Windows, Linux/UNIX, macOS and other operating systems. Browse the release notes and **download** pre-releases, Docker images and join the survey.

Download the latest version for Windows

Download Python 3.13.1

Configuration – find python location in C: copy path and paste in environmental variables



Search – env – advanced – Environmental Variables – sys –path - edit – add new – paste –ok – ok

<https://www.jetbrains.com/pycharm/download/?section=windows> Download and Install using PyCharm IDE

Download community version and Install

Simply create project

Make a directory and then create a python file with the name of Demo



=====

Print Statement

- Printing text: we can use "" double quotes or single
- Printing numbers: we use nothing with numbers
- Printing in same line using **end=""**

```
print('1st Lesson')
print("My name is",end=" ")
print("Popal Furqani.")
print(1,end="")
print(2,end="")
print(3)
```

Variables

Variable is a container which stores the data. Variable is nothing but a name given to a reserved memory location where the data is stored.

- We don't need to specify the Data type for any variable
- Printing variables
- Storing different type of data in variable
- We can write variable name in snake case as well (my_first_name = "Popal")
- Printing text and variables in print statement by separating with comma

```
''' this is multiple lines comment Variables in Python'''
```

```
a = 9          # initializing value to variable a
```

```
c = "Popal Furgani"
```

```
print(c)
```

```
b = 23.3
```

```
print(b)
```

```
d = 'Python programmings'
```

```
print(d)
```

```
A,B,C = 1,2,3 # we define multiple variables in same line
```

```
print(A,B,C)  # print all in same line
```

```
a = 10        # updating the value
```

```
print(a)
```

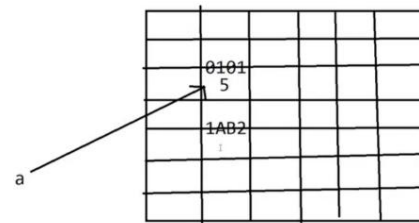
```
A = 7         # A variable is different from a variable in variable name it is case-sensitive
```

```
print(A)
```

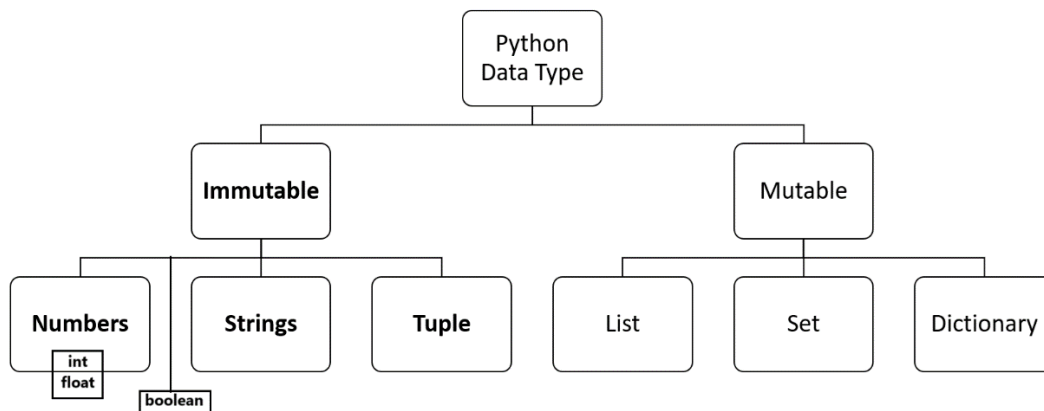
```
x=y=z = 3
```

```
print(x,y,z)  # 3 3 3 if variable has same value
```

```
print("My name is", c)
```



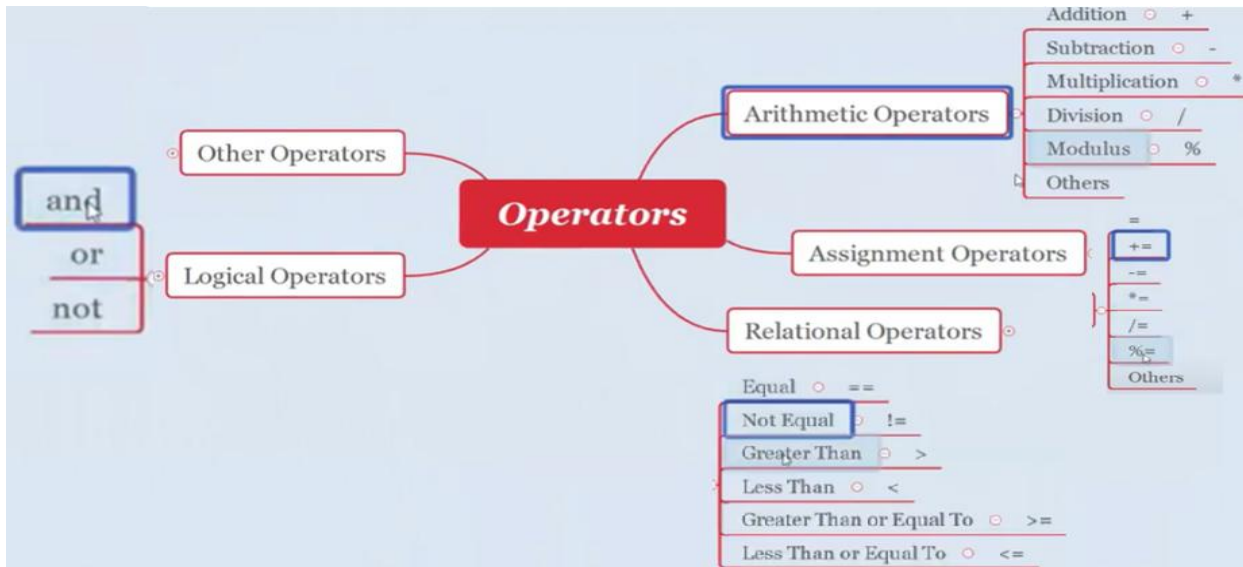
Data Types in Python



Using **type()** for finding Data Type of the variable

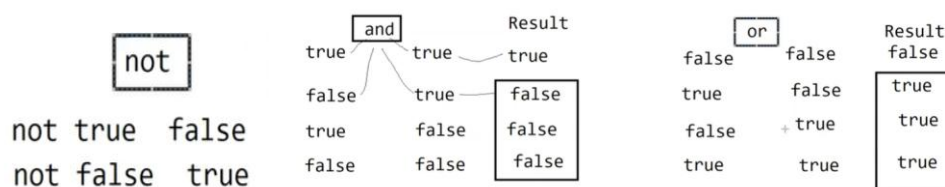
```
'''Data types'''
a = 9
b = 3.2
c = True
print(type(c)) # bool
'''Type Casting'''
x = int("9")
print(type(x))
print(x) # str converted into int
z = float(9)
print(type(z))
print(z) # int converted into float
y = str(9)
print(type(y))
print(y) # int converted into str
w = int("Tony") # You will get error not possible
```

Operators



A = 3+5 => (3 and 5 are operands)

```
a = 5
b = 4
print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a//b) # Floor Division ignores decimal parts
print(a**b) # power
print(a>b)
print(True and True)
```



Comments

- **Single Line Comments in Python #**
- **Multi-Line Comments in Python using `"""` `"""`**
- **Purpose of using Comments: to explain codes to others, to make tips for yourself, disable the code not to execute. Practical Demonstration**

Operator Precedence

- **()** have the highest priority in python
- Followed by ****** (Exponential operator)
- Followed by ***/%//** (have the same precedence)

- Followed by + - (have the same precedence)
- Same precedence operators are evaluated from left to right

Delete a Variable

```
a = 12
print(a)
del a
print(a)
```

String Concatenation in Print Statement

- Using + sign for String concatenation
 - + can only concatenate String text with another String text
 - + cannot be used to concatenate String text with a number
- Using , sign in print statements when concatenating string text with numbers

```
print("Popal"+" " + "Furqani") # Contenation Operator
print(4+5) # Addition Operator
print("Popal" + str(9))
print("Popal" + 9) # Type Error
print("Popal", 9)
```

Storing Multi-line preformatted String text into a variable (Python Tutorial - Part 13)

- We have to use """ for storing the multi-line pre-formatted String text into a variable in Python
- Practical Demonstration

```
p = """
Name:      Ahmad
Phone:     9484848
"""
print(p)
```

Storing a lengthy text into a variable (Python Tutorial - Part 14)

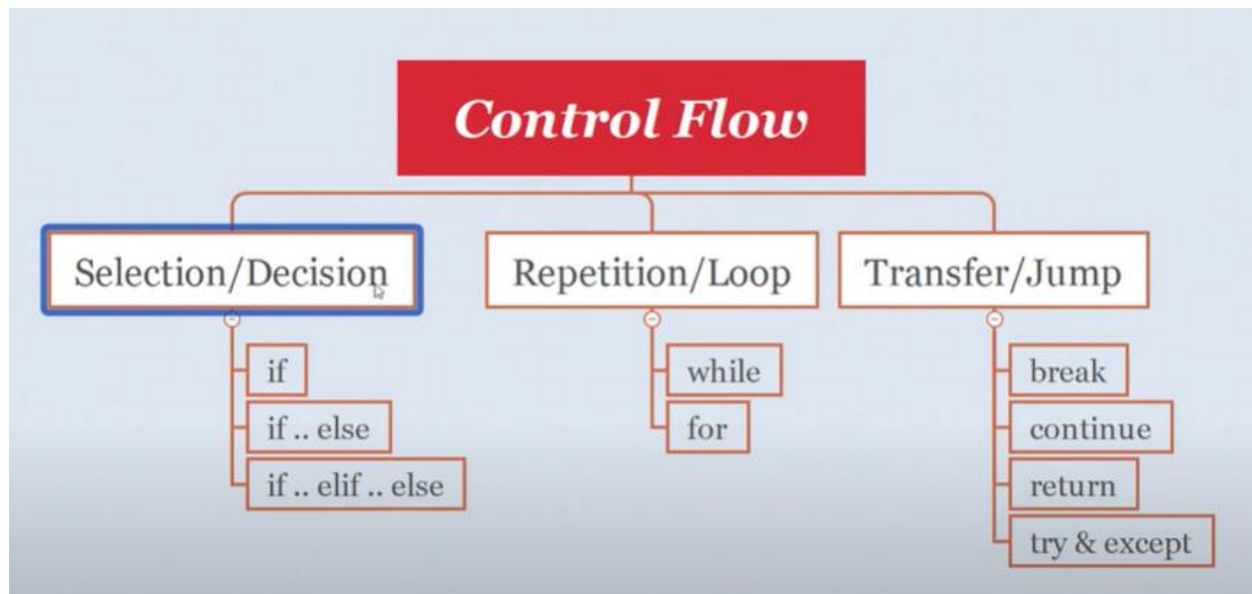
- Practical Demonstration of storing a lengthy text into a variable in Python using \

```
a = ("Hello my name is Gulam Furgani." \
    " I am software engineer. " \
    "I am Teaching Java for beginners ")
print(a) # result will be all in same line
b = ("Hello my name is Gulam Furgani.\n"
    " I am software engineer. \n"
    "I am Teaching Java for beginners ")
print(b) # result will be in new lines
```

Formatting Print statements in Python (Python Tutorial - Part 15)

- Using + and str()
- Using ,
- Using {}
- Using {index}
- Using %s,%d,%f or %g

```
name = "Popal furgani"
experience = 12
location = "VA"
print("My name is "+name+" I have " + str(experience)+" Years of Exp and I stay in "+location)
print("My name is "+name+" I have" , experience ,"Years of Exp and I stay in "+location)
print("My name is {} I have {} Years of Exp and I stay in {}".format(*args: name,experience,location))
print("My name is {2} I have {1} Years of Exp and I stay in {0} ".format(*args: location,experience,name)) # fix index format
```



Selection/Decision Control Statements

Selection/Decision Control Statements in Python - if, else, elif (Python Tutorial - Part 17)

- Decision making in Python using if statements and indentation if a>b:
- if .. else statements in Python - if a>b: else:
- elif statements in Python if a>b: elif b>c: else:

```
""" if """
a = 4
b = 5
if a>b:
    print(" a is greater than b")
    print("a>b")

print("End of the program")

""" if else """
c = 4
d = 5
if c<d:
    print(" c is greater than d")
else:
    print("d is greater than c")
print("End of the program")
```

```

""" if elif else """
a,b,c = 5,8,2
if a>b and a>c:
    print(" a is greatest number")
elif b>a and b>c:
    print("b is greatest number")
else:
    print(" c is the greatest")
print("End of the program")

```

```

""" if elif else """
light = "red"
if light == "red":
    print("Stop")
elif light == "green":
    print("Go")
elif light == "yellow":
    print("Slow down")
else:
    print("Invalid light color")
print("End of the program")

```

Repetition Loop:

while loop in Python (Python Tutorial - Part 18)

- while loop in Python
- Example while i<=10:
- while loop won't be executed once, if the condition is false by default
- while loop executes infinite times if the condition is always true
- while loop iterates until the condition becomes false
- Using else block with while loop
- Example: while i<10: else:

```

# Initialize i to 1 for counting upwards
i = 1
# Start while loop to count from 1 to 10
while i <= 10:
    print(i) # Print current value of i
    i += 1 # Increment i by 1 to move towards condition becoming false
else:
    # Once condition i <= 10 becomes false, this block will execute
    print("While come out of execution when condition become false", i)
# After the loop ends, print a message indicating the end of the program
print("End of Program")

```

```

# Initialize i to 10 for counting downwards
i = 10
# Start while loop to count from 10 to 1
while i >= 1:
    print(i) # Print current value of i
    i -= 1 # Decrement i by 1 to move towards condition becoming false
else:
    # Once condition i >= 1 becomes false, this block will execute
    print("While come out of execution when condition become false", i)
# After the loop ends, print a message indicating the end of the program
print("End of Program")

```

for loop with range() in Python (Python Tutorial - Part 19)

- Using for loop - for i in range(1,11): - else: block with for loop
- for loop with range()
 - range(10)
 - range(3,8)
 - range(2,10,2)
 - range(10,1,-1)

```

for i in range(10): # 0 ...9
    print(i)

for a in range(1,11): # 1 ...10
    print(a)

for b in range(10,0,-1): # 10 ...1
    print(b)

for c in range(1,11,2): # 1..3.5.7.9
    print(c)

```

break and continue statements in Python - Transfer/Jump Statements (Python Tutorial - Part 20)

- break statement in Python
- continue statement in Python
- Using break and continue statements with for loop
- Using break and continue statements with while loop

```

for i in range(1,11):
    if i==5:
        break
    print(i)

for a in range(1,11):
    if a==5:
        continue # when a value becomes 5 wil skip
    print(a)

```

Functions:

In Python, a **function is a block of reusable code** that performs a specific task like method in JAVA. Functions allow you to organize your code, make it more modular, and avoid repetition. You can define a function once and call it whenever you need it. Functions can take inputs (called parameters or arguments) and can return outputs (results).

The keyword `def` is used to define a function. It marks the beginning of the function definition. After `def`, you specify the function name and the parameters (if any), followed by a colon (`:`). The body of the function is indented under the `def` line.

If we were to translate the formula $f(x) = 2x + 3$ into a Python function, it would look like this:

python

 Copy

```

def f(x):
    return 2 * x + 3

```

How it works:

- When you call `f(5)`, Python will calculate `2 * 5 + 3`, which equals `13`.

python

Copy

```
def add_numbers(a, b):  
    return a + b  
  
# Call the function with arguments 3 and 5  
result = add_numbers(3, 5)  
  
print(result) # Output will be 8
```

In this example:

- `add_numbers` is the function name.
- `a` and `b` are parameters.
- The function returns the sum of `a` and `b`.
- The `print` statement outputs the result when the function is called.

```
def sum(a,b): # define 1 usage  
    return a+b  
  
result = sum(a: 12, b: 12) # call  
  
print(result) # print/output
```

Functions - Getting started (Python Tutorial - Part 21)

- Function is a block of code
- `def my_first_function():`
- Calling function to execute the block of code

Function can be called multiple times (Python Tutorial - Part 22)

- Same function can be called multiple times
- Practical Demonstration

```
def first_name(): # define a function 1 usage
    print("My name is Popal Furqani")

first_name() # call a function
```

Parameterizing functions (Python Tutorial - Part 23)

- Functions can intake data
- Example: Function intaking the name
- Practical Demonstration

```
def print_name(name): # name is parameter 3 usages
    print("Your name is: " + name)

print_name("Popal") # Popal is Argument
print_name("Furqani")
print_name("Khan")
```

Default Arguments in functions (Python Tutorial - Part 24)

- Default Arguments in functions
- def function_name(name="Arun Motoori");

```
"""Default Argument"""
def print_name(name="Popal"): # name is parameter 3 usages
    print("Your name is: " + name)

print_name() # Popal is default Argument
print_name("Furqani")
print_name("Ershad")
```

Function with multiple parameters (Python Tutorial - Part 25)

- Function with multiple parameters
- Example: `def function_name(a,b,c)`

```
"""Multiple Parameters"""  
def sum(a,b,c,d): # name is parameter 1 usage  
    print(a+b+c+d)  
  
sum(a: 12, b: 12, c: 12, d: 12) # provide same numbers of Args output will be 48
```

Functions can return data (Python Tutorial - Part 26)

- Function returning data
- Example: return sum
- Practical Demonstration

```
"""Multiple Parameters with Return data"""  
def sum(a,b): 1 usage  
    return a+b  
result = sum(a: 12, b: 12)  
print(result)
```

Purpose of functions (Python Tutorial - Part 27)

Skip countdown

- Repetition of the code can be resolved using functions
- Practical Demonstration

"""Purpose of Function to avoid repetition of code"""

```
def sum(a,b): 3 usages
```

```
    s = a+b
```

```
    print(s)
```

```
sum( a: 1, b: 2)
```

```
sum( a: 4, b: 6)
```

```
sum( a: 9, b: 8)
```

```
a = 5
b = 4
sum = a+b
print(sum)

c = 3
d = 5
sum = c+d
print(sum)

e = 2
f = 4
sum = e+f
print(sum)
```

Multiple functions can be created (Python Tutorial - Part 28)

- We can create multiple functions in the same python file
- Practical Demonstration

```
def sum(a,b): 1 usage
```

```
    print(a+b)
```

```
def sub(a,b): 1 usage
```

```
    print(a-b)
```

```
def mul(a,b): 1 usage
```

```
    print(a*b)
```

```
def div(a,b): 1 usage
```

```
    print(a/b)
```

```
sum( a: 10, b: 10) #20
```

```
sub( a: 10, b: 10) #0
```

```
mul( a: 10, b: 10) #100
```

```
div( a: 10, b: 10) #1.0
```

Built-in Functions

Input () allow user to enter data from console

Max and Min for maximum and minimum values

Collecting input from User using input() in-built function (Python Tutorial - Part 29)

- Collecting input from User and Type Casting if required
- input() in-built function
- Practical Demonstration

```
""" Input Function return String Output """
print("Please Enter Your Name Here:")
name = input()
print("Welcome " + name)
""" Input Function for Integer value """
print("Enter 1st number")
num1 = input()
print("Enter 2nd Number")
num2=input()
print("Multiplication of the given numbers is:"+str (int(num1)*int (num2)))
print("Multiplication of the given numbers is:", (int(num1)*int (num2)))
```

max() and min() in-built functions in python (Python Tutorial - Part 30)

- max() and min() in-built functions in python
 - x = max(8,33,1,9,47,54,12,6,3)
 - y = min(8,33,1,9,47,54,12,6,3)
 - Also, you can use with string text
- Practical Demonstration

```
ma = max(12,3,5,1,3,6,4,2,9)
print(ma)
mi = min(12,3,5,1,3,6,4,2,9)
print(mi)

mi = min("Red", "Bluef", "Orange")
print(mi)
```

Local and Global Variables in Python

(Python Tutorial - Part 31)

- Local and Global Variables in Python

- **Global Variables** - Variable created directly in the Python file and outside the functions - [View Code Here](#)
- **Local Variables** - Variable created inside the function - [View Code Here](#)
- Practical Demonstration of having both Global Variable and Local Variable in same python file - [View Code Here](#)
- Creating Global Variable inside the function using **global** keyword - [View Code Here](#)

Local & Global Variables:

```
def my_function(): 1 usage
    name = "Popal furqani" # Local Variable
    print(name)

my_function()
print(name) # Not accessible outside function
```

```
name = "Popal furqani" # Global Variable
def my_function(): 1 usage
    print(name) # accessible inside the function

my_function()
print(name) # accessible outside the function
```

```
def my_function(): 1 usage
    global name
    name = "Popal furqani" # Global Variable
    print(name) # accessible inside the function

my_function()
print(name) # accessible outside the function
```

```

name = "Gulam Furgani" # Global Variable
def my_function(): 1 usage
    name= "Popal furgani" # Local Variable
    print(name) # only local variable accessed

my_function()
print(name) # Global variable accessed

```

```

name = "Gulam Furgani" # Global Variable
def my_function(): 1 usage
    global name
    name= "Popal furgani" # updating the value of global variable
    print(name) # Popal Furgani

my_function()
print(name) # Global variable accessed # Popal Furgani

```

pass statement in Python (Python Tutorial - Part 32)

- pass statement in Python
 - If the function or loop or condition block in Python program is not implemented yet and will be implemented later, we simply add pass under it for future implementation
 - When we execute the functions having pass under a function or loop, nothing happens.
 - Examples:
 - function having pass
 - def my_fun():

```

def sum(): 1 usage
    pass

sum() # if call it nothing will print

for i in range(1,11):
    pass

```

Collections in Python (Python Tutorial - Part 33)

- Collections in Python
 - Collection Data Types - List, Tuple, Set and Dictionary

Collections:

Which allow us to store multiple VALUES INTO a single variable

There are FOUR collection DATA TYPES IN PYTHON

- List
- Tuple
- Set
- DICTIONARY

List:

```
#list
colors = ["orange", "green", "yellow", "red", "white"]
print(type(colors))
```

The following topics are covered in this session:

- What is a List?
- List Collection Data Type
- Printing the List
- Using index to retrieve the elements in the list
- Finding the size of the list using len() function
- Using for loop with range() with list
- Using for-each loop with in operator with list
- Updating the existing elements in the list
- Appending the elements to the end of the list using append() function
- Inserting the elements at a particular index of the list using insert() function
- Removing the elements from the List using remove() function
- Removing the last element from the List using pop() function
- Removing the elements using their index using pop(index) function
- Removing all the elements in the list using clear() function
- Reversing the elements in the list using reverse() function
- Sorting the elements of the list using sort() function
- Using index() function with list
- Nested List
- Storing different types of values into the same list

- Using Forward index and Backward index with list
- Slicing Lists
- Finding the number of times the element is repeated in the list using count() function
- Finding the maximum and minimum elements in the list using max() and min() functions
- Finding the sum of the elements in the list using sum() function
- Finding the data type of the list using type() function
- Deleting the list using del keyword
- Using in and not in operators with list
- Concatenating two lists using + symbol in Python
- Extending the list with another list using extend() function

```

a =[8,6,5,4]
print(a)
a =[8,6,5,4] # indexing
print(a[2]) # 5 is in index 2
print(a[0]) # 8
print(a[1]) # 6
""" size of list """
len(a)
print(len(a))
for i in range(len(a)): # loop with list
    print(a[i]) # 0-3
# for each loop (print each and every element in the list which is a)
for e in a:
    print(e)
# updating value of list
a[2] = 9 # 5 will be updated to 9 in a list
# append to add value
a.append(1)
print(a) # [8,6,9,4,1]
# insert value
a.insert( __index:2, __object 10)
print(a) # [8,6,10,9,4,1]
# remove elements
a.remove(10)
a.pop() # delete the last element in list
a.pop(2) # delete element in index 2
# a.clear() # remove all values form list
# reverse list
a.reverse()

```

```

# sort list str will a-z and numbers lower to upper
a.sort()
print(a)
# index of an element
# print(a.index(10))
# nested list
a = [5,9,3,[2,2,3,5],11]
print(a[1])
# Multiple values in a single variable
b = ["Honda", "Oddeysai", 2016, 90000]
print(b[2]) # print(b)
c = [5,9,3,2,2,3,5,11,9,3,2,2,3,5,11]
print(c.count(2))
print(max(c))
print(min(c))
print(sum(c))
print(type(c)) # data type of c variable is list
# del c # delete the list
# concatenate lists
d = a+b
print(d)
a.extend(b)
print(a) # a list will be extended with b list

```

Tuple:

Tuple - Collections (Python Tutorial - Part 35)

- Tuple
 - Tuple is one of the four Collection data types in Python
 - Create Tuple using ()
 - Print Tuple
 - a = (5,1,3,4,8)
 - print(a)
 - print(type(a))
 - Access values in Tuple
 - Forward Index
 - Backward Index
 - Tuples are immutable

List	Tuple
Mutable can be updated / modified	Immutable : can't be updated / modified
[]	()
	No append , No pop, no remove,

Everything is the same

```
a = (4,5,9,5,"Ali")
print(a)
print(a[2]) # index of 2
print(len(a))
```

```
a =(8,6,5,4)
print(a)
print(a[2]) # 5 is in index 2
print(a[0]) # 8
print(a[1]) # 6
""" size of Tuple """
len(a)
print(len(a))
for i in range(len(a)): # loop with Tuple
    print(a[i]) # 0-3
# for each loop (print each and every element in the tuple which is a)
for e in a:
    print(e)
# updating value of tuple is not possible
# append to add value is not possible with tuple
# insert value
a.insert(2,10)
print(a) # [8,6,10,9,4,1]
# remove elements not possible
# a.clear() # remove all values form tuple
# reverse tuple
a.reverse()
# sort list str will a-z and numbers lower to upper
a.sort()
print(a)
# nested tuple
a = (5,9,3,(2,2,3,5),11)
print(a[1])
# Multiple values in a single variable
b = ("Honda","Oddeyssi",2016,90000)
print(b[2]) # print(b)

print(b[2]) # print(b)
c = [5,9,3,2,2,3,5,11,9,3,2,2,3,5,11]
print(c.count(2))
print(max(c))
print(min(c))
print(sum(c))
print(type(c)) # data type of c variable is list
# del c # delete the tuple
```

Set:

Set - Collections (Python Tutorial - Part 36)

- Set
 - Set is one of the Collection Data types in Python
 - Create Set using {}
 - Un-ordered
 - Demonstrate this by printing it
 - Un-indexed
 - Un-Duplicated
 - Printing Set
 - Find the type of Set
 - Find the length of Set - len()

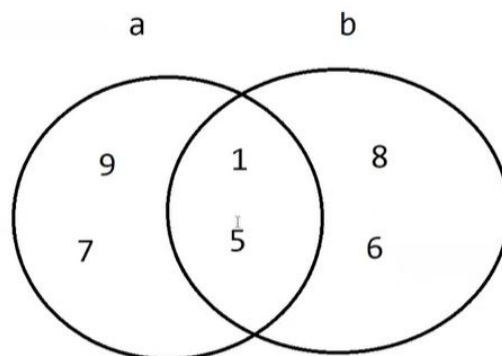
List	Tuple	Set
[]	()	{ }
Mutable	Immutable	Mutable
Indexing	Indexing	No-Index
Duplicates accepted	Duplicates accepted	Duplicates are ignored
Element stored in order	//	elements are stored in random

- Removing the values from Set without knowing where it is available in the set
 - `discard(value)`
 - Randomly removing the value from the Set
 - Using `pop()`
 - Removing all the values at a go from the Set
 - Using `clear()`
 - Deleting the Set
 - `del set_name`
 - Using for loop with Set
 - for each loop
 - for loop with `range()` - Not possible
 - Using in and not in operators with Set
 - Set can nest only Tuple
-
- Using for loop with Set
 - for each loop
 - for loop with `range()` - Not possible
 - Using in and not in operators with Set
 - Set can nest only Tuple
 - Set cannot nest List and Set
 - Converting a List to a Set
 - `set(list_name)`
 - Union
 - `print(a.union(b))`
 - `print(a | b)`
 - Intersection
 - `print(a.intersection(b))`

difference

$a - b$
 $b - a$

Symmetric
 difference



Union

All

Intersection

Common

```

a = {9,1,5,7,9}
print(a) # 9 5 1 7 different order and last 9 ignored
print(type(a))
a.add(8)

a.update([9,12])
print(a)
# a.clear()

# for loop, we cant use, but we can use for each loop
for e in a:
    print(e)
# union
print(a)
b = {1,5,6}
print(a.union(b))
print(a | b) # is shortcut for union all
print(a.difference(b))
print(max(a))
print(sum(a))

```

Dictionary:

Dictionary - Collections (Python Tutorial - Part 37)

- Dictionary
 - Dictionary is one of the Collection Data Types in Python
 - Dictionary stores the multiple values in Key Value pairs
 - Create Dictionary using {Key:Value,Key:Value,Key:Value}
 - Dictionary and indentation
 - Print Dictionary
 - Get the value of a Key
 - dictionary_name.get(Key)
 - dictionary_name[Key]
 - Printing all the keys of the Dictionary

- Printing all the keys of the Dictionary
 - dictionary_name.keys()
- Printing all the values of the Dictionary
 - dictionary_name.values()
- Updating the values of the Dictionary
 - dictionary_name[Key] = NewValue
 - Print dictionary
- Adding new Key, Values to the existing Dictionary
 - dictionary_name[NewKey] = NewValue
 - Print dictionary

- Using for loop with dictionary
 - for k in dictionary_name:
 - print(k)
 - for k in dictionary_name:
 - print(k)
 - print(dictionary_name[k])
 - for k in dictionary_name:
 - print(k,":",dictionary_name[k])
 - for v in dictionary_name.values():
 - print(v)
 - for k,v in dictionary_name.items():
 - print(k,v)
- Other Functions we can use with Dictionary
 - Removing an item having the specific key from Dictionary and returns the value of the removed key
 - dictionary_name.pop(Key)
 - del dictionary_name[Key]
 - Removing the last item in the Dictionary and returns the removed element
 - dictionary_name.popitem()
 - Remove all the key,values from the Dictionary at a go
 - dictionary_name.clear()
 - Find the size of the dictionary
 - len(dictionary_name)
 - Deleting the complete dictionary
 - del dictionary_name
- Comparing Dictionary using == and !=

List	Tuple	Set	Dictionary
Group of values			Group of Key, Value pairs
["orange", "red", "blue"]			{ "Brand": "Honda", "Model": "Amaze", "Price": 900000, "Milage": 14.5 }

```

car = {"Brand": "Honda", "Modle": "Civic", "price": 50000, "Milage": 1400}
print(car)
print(car.keys())
print(car.values())
car["price"] = 80000 # update
print(car)
# add new key
car["color"] = "blue"
print(car)
# loop + Dic
# for k in car.keys():
#     print(k)
# for v in car.values():
#     print(v)
# for e in car.keys():
#     print(e, car[e])
for k, v in car.items():
    print(k, v)
del car["Milage"]
print(car)
# car.clear() # clear everything in dic or del car
car1 = {"Brand": "Honda", "Modle": "Civic", "price": 50000, "Milage": 1400}
print(car == car1)

```

Difference between collection:

List Versus Tuple Versus Set Versus Dictionary (Python Tutorial - Part 38)

- List Versus Tuple Versus Set Versus Dictionary
 - List - Indexing - Stores Elements in order - Allows Duplicate Elements - Mutable - []
 - Tuple - Indexing - Stores Elements in order - Allows Duplicate Elements - Immutable - ()
 - Set - No Indexing - Randomly stores the elements - Duplicate Elements are ignored - Mutable {}
 - Dictionary - Key Value pairs - Unique Keys can have duplicate values - Mutable {}

List	Tuple	Set	Dictionary
[v1,v2,v3]	(v1,v2,v3)	{v1,v2,v3}	{k1:v1,k2:v2,k3:v3}
Indexing	Indexing	No Indexing	No Indexing
Same order	Same order	Random order	Same order
Mutable	Immutable	Mutable	Mutable
Duplicates Accepted	Duplicates Accepted	Duplicated ignored	Keys are unique Values can be Duplicated

Strings:

Strings (Python Tutorial - Part 39)

- Strings
 - In python, you can store Strings into a variable by surrounding the text with single quotes or double quotes
 - Examples:
 - name = "Arun Motoori"
 - location = 'Hyderabad'
 - color = str("black")

- Strings work like Collection List in Python

- Examples:

- print(name[0])
 - for letter in name:
 - print(len(name))
 - print("run" in name)
 - print("run" not in name)

```
name="Furqani" # each letter acts as value
# print(name[0])
# print(name[1])
# print(name[2])
for e in name:
    print(e)
print(len(name)) # size of string
for i in range(len(name)):
    print(name[i])
print('ni' in name)
```

- Slicing Strings

- print(name[1:4]) - Index 1 to position 4
 - print(name[:4]) - Index 0 to position 4
 - print(name[1:]) - Index 1 to last position

- Modifying Strings

- print(name.upper())
 - print(name.lower())
 - print(" Arun Motoori ".strip()) - Removing leading and trailing spaces
 - print(name.replace("n","b"))
 - print(name.split(" "))

- in and not in operator with String

- print("run" in "Arun")

```
name="My-name-is-Popal-Furqani" # slicing
print(name[1:5])
print(name.upper())
print(name.lower())
print(name.strip())
print(name.replace(_old: "u", _new: "o"))
# slit nmae into sub string
words = name.split("-") # by space " ", by ".", by "-"
for e in words:
    print(e)
```

- Strings are immutable in Python

- name = "Arun"
- print(id(name))
- name = "Motoori"
- print(id(name))

- Other String functions

- name.capitalize() - First letter of the statement will get capitalized
- name.title() - First letter of each word in the statement gets capitalized
- name.count("word") - returns the number of times the give word is available in the string
- a.find("word") - returns index if found, otherwise returns -1
- comparing strings in Python
 - Using ==
 - Using `__eq__()` function - `a.__eq__(b)`

```
name="My name is Popal Furgani My name is Popal Furgani"
name1="My name is Popal Furgani My name is Popal Furgani"
print(name.capitalize())
print(name.title()) # capitalize each word
print(name.count("is")) # 2 times repeated
print(name.find("Popal")) # 11 index, if -1 means not there
print(name == name1) # to compare strings
```

In & not in Operators:

in and not in operators (Python Tutorial - Part 40)

- in and not in operator

- Can be used with:

- List
- Tuple
- Set
- for loop with range()
- for each loop
- Strings

- Practical Demonstration

- Example One:

- colors = ["orange","blue","green"]

- `print("red" in colors)`
- `print("green" in colors)`
- `print("red" not in colors)`
- `print("green" not in colors)`
- Example Two:
 - `colors = ["orange", "blue", "green"]`
 - `for color in colors:`
 - `print(color)`

```
colors = ["red", "green", "blue", "Yellow"]
print("red" in colors)
for i in range(11):
    print(i)
for e in colors:
    print(e)
name = "My name is Popal Furgani"
print("Popal" in name)
```

File Handling:

What is a **file** in Python?

file is just a variable name used to store the file object returned by the `open ()` function.

A **file** in Python is a way to store data permanently. Python allows us to read, write, and manipulate files stored on a computer.

Understanding `open ()` in Python

- The `open ()` function is a built-in function in Python that is used to open a file and return a **file object**.
- This file object allows us to perform operations like **reading, writing, and closing** the file.

Types of File Operations in Python

1. **Opening a File** → `open(filename, mode)`
2. **Reading a File** → `.read()`, `.readline()`, `.readlines()`
3. **Writing to a File** → `.write()`, `.writelines()`
4. **Closing a File** → `.close()`

File Opening Modes

Mode	Description
"r"	Read (default mode, file must exist)
"w"	Write (creates a new file or overwrites an existing one)
"a"	Append (adds data to the file without overwriting)
"x"	Create (creates a new file, but fails if it already exists)
"rb"	Read in binary mode
"wb"	Write in binary mode

File Handling (Python Tutorial - Part 41)

- File Handling
 - Reading data from Files and Writing data into Files using Python.
 - Opening, Writing and Closing Files
 - We can open the Files in read only (r), write (w) and appending (a) modes
 - `file = open("C:\\abc\\xyz.txt", "w")`
 - `file.write("My name is Arun Motoori \n")`
 - `file.write("I am teaching Python Programming")`
 - `file.close()`

Open – write – close

```
# Open path in write mode, write and close
file = open("C:\\Users\\qazif\\PycharmProjects\\PythonProject\\Files\\popal.txt", "w")

file.write("My name is Popal Furgani\n")
file.write("I am teaching Python programming language")

file.close()
```

- Reading Files

- file = open("C:\\abc\\xyz.txt","r")
- print(file.read()) - All characters will be read here
- print(file.read(15)) - 15 characters will be read here
- print(file.readlines()) - Reads all the lines in the list format
- print(file.readline()) - Reads line by line but only one line
- file.close()

- Appending text to existing file content

- file = open("C:\\abc\\xyz.txt","a")
- file.write("\nPython is an easy programming language")
- file.close()

```
# Open path in read mode, read and close
file = open("C:\\Users\\qazif\\PycharmProjects\\PythonProject\\Files\\popal.txt","r")

print(file.read())

file.close()
```

```
# Open path in append mode, add and close
file = open("C:\\Users\\qazif\\PycharmProjects\\PythonProject\\Files\\popal.txt","a")

file.write("Python is an easy language")

file.close()
```

Using for each loop for reading a file

- file = open("C:\\abc\\xyz.txt","r")
- for line in file:
 - print(line)

```
# Open , for loop and close to read each and every line in file
file = open("C:\\Users\\qazif\\PycharmProjects\\PythonProject\\Files\\popal.txt","r")

for line in file:
    print(line)

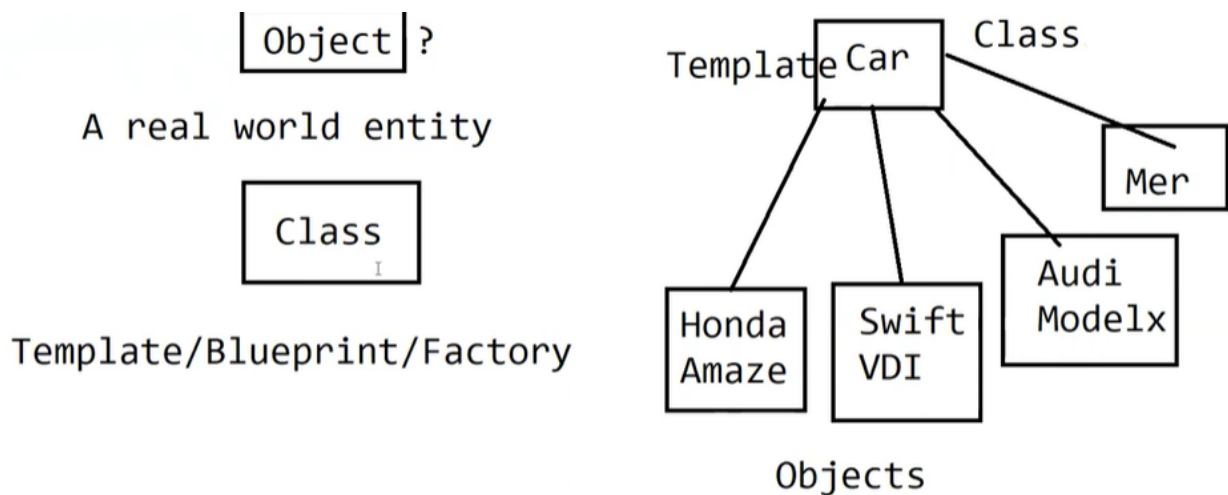
file.close()
```

Class and Object:

Getting started with Object Oriented Programming - Classes and Objects (Python Tutorial - Part 42)

- Object is a real world entity
- Examples for real world entities
- We can represent real world entities as Objects in Python
- Classes in Python, can be used for creating these Objects
- Create Class in Python - class Car:
- Creating variables and functions inside the class
- Functions inside the class in Python can be called as methods
- self keyword will be explained later
- Creating Object for a Class - Car()
- Using object reference for accessing the object memory of the class - car1 = Car()

- Using object reference to access the variables and methods of the class



```

# The below is a function in python
def sample_fun():
    print("Inside sample function")

class Car:

    wheels = 4

    # The below is a method in Python
    def start_car(self):
        print("Car started")

```

```

class Car: 2 usages
    wheels = 4 # variable
    def start_car(self): # Method of in class 2 usages
        print("Car started")
# object creation of the class/ hamaze is object reference
hamaze = Car()

# to access variable and methods we use object reference
print(hamaze.wheels)
hamaze.start_car()

honda = Car() # second object creation
print(honda.wheels)
honda.start_car()

```

Self keyword:

self in Python (Python Tutorial - Part 43)

- self in Python
 - Represents something that belongs to the class
 - Can be used to access the variables and methods inside the same class
 - Inside the Class we have to use self to access the properties and outside the class we have to use object reference to access the class properties
 - self parameter can be renamed
 - Assigning function parameters to class variables (Will be explained in the next session in detail)

```

class Car: 1 usage
    wheels = 4

    def Start_car(self): 1 usage
        print("Car has been started")
        print(self.wheels)

    # variable and methods are properties of the class
    # using self keyword we can access properties inside the class
    # using object reference we can access properties outside the class
car1 = Car()
print(car1.wheels)
car1.Start_car()

```

Assigning method parameters to class variables using self keyword (Python Tutorial - Part 44)

- We can use self keyword in python to differentiate class variables from the method parameters
- Practical Demonstration

```

class Car: 1 usage
    wheels = 4
    def sample(self, brand, price, milage): 1 usage
        self.brand = brand # this variable becomes class level by using self key word
        self.price = price
        self.milage = milage
        print(brand, price, milage)

    def sample2(self): 1 usage
        print(self.wheels, self.brand, self.price, self.milage)

car1 = Car()
car1.sample( brand: "Honda", price: 4000, milage: 12000)
car1.sample2()

```

Initializing Class Variables using methods in Python (Python Tutorial - Part 45)

- Create a Car class and different objects
- Using a method inside Car class to initialize the class variables
- Practical Demonstration

```
class Car: 2 usages

    def ini_method(self, brand, model, price, milage): 2 usages
        self.brand = brand
        self.model = model
        self.price = price
        self.milage = milage

    def start_car(self): 2 usages
        print(self.brand + self.model + str(self.price) + str(self.milage) + "Car has started")

honda=Car() # Object1 creation in the memory with the reference name of honda
honda.ini_method( brand: "Honda ", model: "civic ", price: 60000, milage: 50000) # initialization process
toyota=Car() # Object2 creation in the memory with the reference name of toyota
toyota.ini_method( brand: "Toyota ", model: "corolla ", price: 12000, milage: 45000) # initialization process
honda.start_car()
toyota.start_car()
```

__init__ method in Python (Python Tutorial - Part 46)

Skip countdown

- We can initialize the Class variables during the object creation statement itself
- Hence initialization process is simplified with __init__ method
- Using __init__ method inside Car class to initialize the class variables
- Practical Demonstration

```

class Car: 2 usages

    def __init__(self, brand, model, price, milage): # init method (works as constructor in java)
        self.brand = brand
        self.model = model
        self.price = price
        self.milage = milage

    def start_car(self): 2 usages
        print(self.brand + self.model+ str(self.price)+str(self.milage) + "Car has started")

honda=Car( brand:"Honda ", model:"civic ", price:60000, milage:50000) # Object1 creation in the memory with the reference name of honda
toyota=Car( brand:"Toyota ", model:"corolla ", price:12000, milage:45000) # Object2 creation in the memory with the reference name of to
honda.start_car()
toyota.start_car()

```

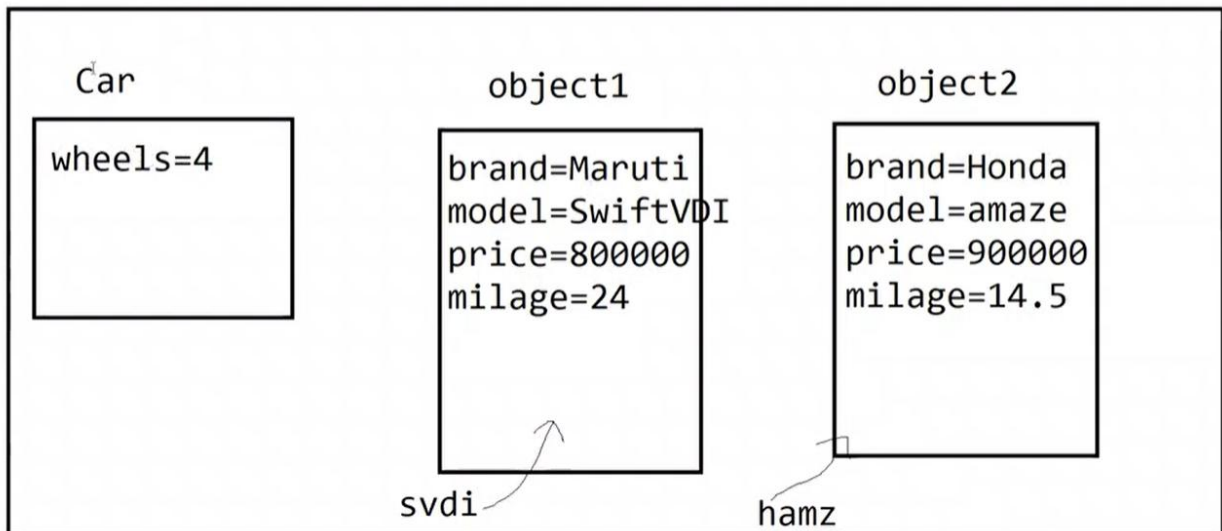
Static and Instance Variable and Methods:

Static Variables, Static Methods, Instance Variables, Instance Methods (Python Tutorial - Part 47)

- Accessing Static Variables using Class Name
- Instance Variables in Python
- Methods in Python
 - Functions inside the class in Python can be called as methods
 - Instance methods
 - Needs to be accessed using the object
 - has self keyword specified
 - Static methods
 - @staticmethod
 - should not be specified with self keyword
 - If we provide self keyword in the static methods, that will be treated as a first parameter, for which we have to pass the argument while calling it
 - Can be accessed using the Class name
- Instance Variables and Instance Methods belong to Object Memory
- Static Variables and Static Methods belong to Class Memory
- Static Methods can only access Static Stuff

Instance Related to object and related to class

static	instance
static variables static methods	instance variables instance methods
Class wheels = 4 Common	objects Non-Common color price milage model



```

class Car: 5 usages
    wheels = 4 # static variable common
    def __init__(self, brand, model, price, milage): # init method (works as constructor in java) is also instance
        self.brand = brand
        self.model = model
        self.price = price
        self.milage = milage # instance variable :to access inside the class we use self keyword

    def start_car(self): # instance Method :to access inside the class we use self keyword 2 usages
        print(self.brand + self.model+ str(self.price)+str(self.milage) + "Car has started")

    @staticmethod 1 usage
    def print_car_wheels(): # static method
        print(Car.wheels)

honda=Car( brand:"Honda ", model:"civic ", price:60000, milage:50000) # Object1 creation in the memory with the reference name of honda
toyota=Car( brand:"Toyota ", model:"corolla ", price:12000, milage:45000) # Object2 creation in the memory with the reference name of toyota
honda.start_car() # to access instance method or variable outside the class we use object reference
toyota.start_car()
print(honda.brand) # to access instance method or variable outside the class we use object reference
print(Car.wheels) # to call static method or variable inside or outside the class we use className.Variable-name or method name
Car.print_car_wheels() # to call static method or variable inside or outside the class we use className.Variable-name or method name

```

Inheritance:

Inheritance (Python Tutorial - Part 48)

- Inheritance
 - A Child Class acquiring the properties of Parent Class
 - Practical Demonstration of Inheritance
 - class B(A):

```

class A: 1 usage
    a = 9
    def method_a(self): 1 usage
        print("Parent Class A")

class B(A): # make relationship: B is child - (A) is parent 1 usage
    b = 10
    def method_b(self): 1 usage
        print("Child class B")

obj = B()
print(obj.a) # access parent class variable
print(obj.b) # access child class variable
obj.method_a() # access parent class method
obj.method_b() # access child class method

```

Type of Inheritance:

Types of Inheritance (Python Tutorial - Part 49)

- Types of Inheritance
 - Single
 - Multilevel
 - Hierarchical - Two child classes have same parent class
 - Multiple - A child class has two parent classes - class Three(One,Two):
 - Hybrid - Hierarchical + Multiple

1 Single Inheritance:

```
class A: 1 usage
    a = 9
    def method_a(self): 1 usage
        print("Parent Class A")

class B(A): # make relationship: B is child - (A) is parent 1 usage
    b = 10
    def method_b(self): 1 usage
        print("Child class B")

obj = B()
print(obj.a) # access parent class variable
print(obj.b) # access child class variable
obj.method_a() # access parent class method
obj.method_b() # access child class method
```

2 Multi Level Inheritance:

```

class A: 1 usage
    a = 9
    def method_a(self): 1 usage
        print("Parent Class A")

class B(A): # make relationship: B is child - (A) is parent 1 usage
    b = 10
    def method_b(self): 1 usage
        print("Child class B")

class C(B): # grand-child 1 usage
    c = 11

obj = C()
print(obj.a) # access parent class variable
print(obj.b) # access child class variable
print(obj.c) # access grand-child class variable
obj.method_a() # access parent class method
obj.method_b() # access child class method

```

3 Hierarchical Inheritance:

```

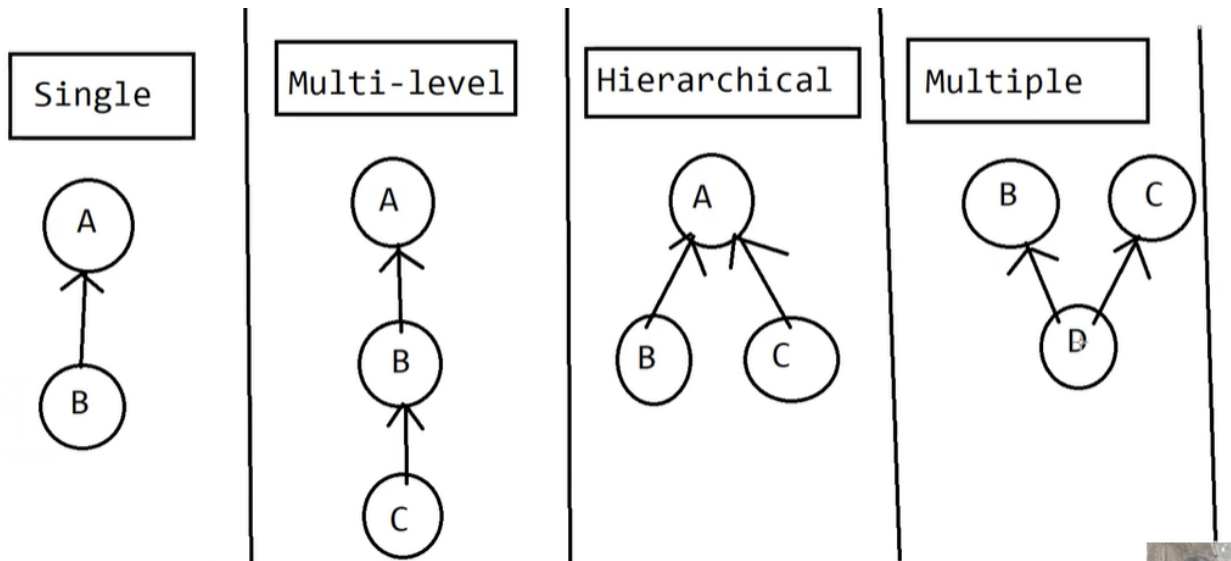
class A: 2 usages
    a = 9
    def method_a(self): 1 usage
        print("Parent Class A")

class B(A): # child 1 make relationship: B is child - (A) is parent
    b = 10
    def method_b(self):
        print("Child class B")

class C(A): # child 2 1 usage
    c = 11

obj = C()
print(obj.a) # access parent class variable
print(obj.b) # access child class variable
print(obj.c) # access grand-child class variable
obj.method_a() # access parent class method
obj.method_b() # access child class method

```



4 Multiple Inheritance:

```

class A: # parent 1 1 usage
    a = 9

class B: # Parent 2 1 usage
    b = 10

class C(A,B): # child 1 usage
    c = 11

obj = C()
print(obj.a) # access parent class variable
print(obj.b) # access parent class variable
print(obj.c) # access child class variable
  
```

5 Hybrid Inheritance: Multiple+ Hierarchical

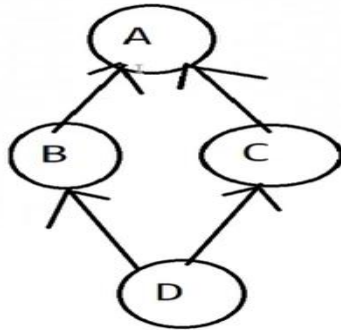
```

class A: # parent of B and C 2 usages
    a = 9
class B(A): # child1 1 usage
    b = 10

class C(A): # child2 1 usage
    c = 11
class D(B,C): # is child of B and C 1 usage
    d = 12
obj = D()
print(obj.a)
print(obj.b)
print(obj.c)

```

Hybrid = Hierarchical + Multiple



Override Method, Variables:

Child class properties will override Parent class properties mean parent class properties will be hidden
override works with inheritance, and same methods in parent and child class.

Polymorphism - Method Overriding (Python Tutorial - Part 50)

- Polymorphism - Method Overriding
 - Overriding variables and methods in Python
- Practical Demonstration

```

class A: 1 usage
    a = 5
    def m1(self):
        print("Parent class")

class B(A): 1 usage
    a = 10
    def m1(self):
        print("Child Class")

obj = B()
print(obj.a) # B class will override A class properties ans = 10

```

Overloading Methods, Variables:

Overloading Methods is not directly supported by Python (Python Tutorial - Part 51)

- Method Overloading in Python is not possible and alternative way to achieve
 - Overloading Methods is not directly supported by Python
 - Alternative method for Method Overloading in Python
 - def methodOne(self, a=None, b=None):
 - if a != None and b != None:
 - print(a*b)
 - elif a != None:

```

class A: 1 usage
    def m1(self): 1 usage
        print("some text1") # first one will be skipped
    def m1(self): 1 usage
        print("some text2")

obj = A()
obj.m1()

```

super() in Python (Python Tutorial - Part 52)

- super()
 - Accessing or Calling the parent class properties (variables and methods) from child class, which are having the same name as child class properties
 - If the child class doesn't have any constructor and the parent class has a constructor, it will be automatically invoked on creating an object for child class
 - But if the child class has its own constructor, then the parent class constructor won't be invoked and we have to call it explicitly from child class constructor using super()

```
class A: 1 usage
    a =5
    def m1(self): 1 usage
        print("Parent A class method")

class B(A): 1 usage
    a = 10
    def m1(self): 1 usage
        print("Child B class method")

    def print_prop(self): 1 usage
        print(self.a) # Child class variable will be called
        self.m1() # Child class method will be called
        print(super().a) # Parent class variable will be called
        super().m1() # Parent class method will be called

obj = B()
obj.print_prop()
```

```
class A: 1 usage
    a =5
    def __init__(self):
        print("Parent A class init method")

class B(A): 1 usage
    def __init__(self):
        super().__init__()
        print("child class init method")

B()
```

Private Variable and Private Methods in Python (Python Tutorial - Part 53)

- Private Variable and Private Methods in Python
 - `__variable_name` or `__method_name`
 - Can be accessed only with in the class
- Practical Demonstration

The only way to access private method outside the class

```
class A: 1 usage
    __a = 5    # private property

    def __m1(self): 1 usage
        print("Private method of A class")

    def printer(self): 1 usage
        print(self.__a)
        self.__m1()

obj = A()
obj.printer()
```

Getter and Setter

Using Getter and Setter Methods with private variables in Python (Python Tutorial - Part 54)

- Getter and Setter non-private methods can be used for accessing the private variables
- Practical Demonstration

```
class A: 3 usages
    __age = 0
    def set_age(self, age): 1 usage
        A.__age = age

    def get_age(self): 1 usage
        return A.__age

obj = A()
obj.set_age(14)
print(obj.get_age())
```

Encapsulation:

Encapsulation in Python (Python Tutorial - Part 55)

- Encapsulation in Python
 - Encapsulation is a mechanism, which wraps the data (i.e. variables) and code (i.e. methods) together under a single unit (i.e. Class)
 - i.e. In Encapsulation, you cannot access data (i.e. variables) as a single unit, instead we have to take the help of methods to access the data (i.e. variables).
 - Encapsulation is achieved by privatizing the variables and accessing them by using getters and setters methods to modify and get the variable values
 - Binding the data to the Class, by making the variables as private and accessing them only via non-private methods (getter and setter methods)
 - Practical Demonstration

Encapsulation

Class Unit

private	non-private
variables +	methods
data	code

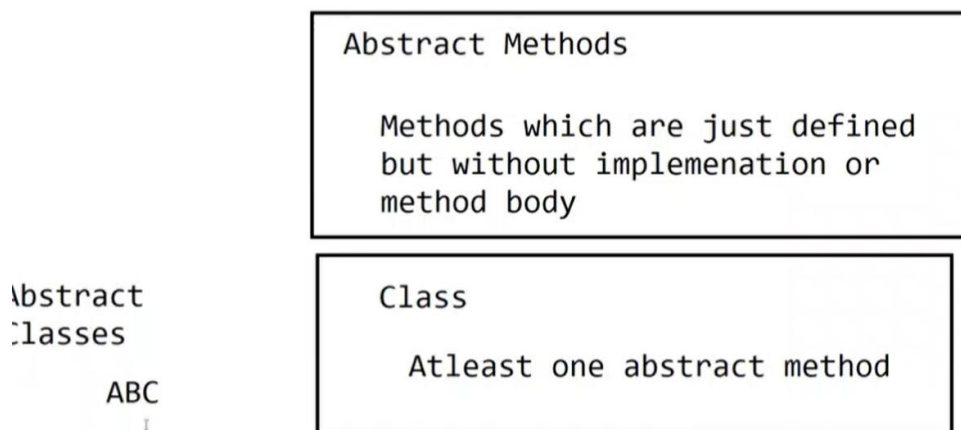
```
class A: # unit 3 usages
    __age = 0 # private variable holds data
    def set_age(self,age): # non-private method holds code 1 usage
        A.__age = age

    def get_age(self): 1 usage
        return A.__age
obj = A()
obj.set_age(14)
print(obj.get_age())
```

Abstraction:

Abstraction in Python - Abstract Classes and Abstract Methods (Python Tutorial - Part 56)

- Abstraction in Python - Abstract Classes and Abstract Methods
 - For partial implementation of Class, we can use Abstract classes
 - When what to implement is clear, but how to implement is not clear, then we have to go for Abstract classes and Abstract methods
 - Creating Abstract Classes and Abstract Methods
- `import abc from ABC, abstractmethod`
- `class A(ABC):`
 - `@abstractmethod`
 - `def fun_one(self):`
 - `pass`
- We cannot create objects for Abstract Classes
- Abstract Methods in the Abstract Class, need to be implemented by its child classes
- We can create object for the child class of abstract class, which has implemented all the methods of the Abstract class
- Child class which is not implementing all the abstract methods of the Parent Abstract class, automatically becomes an Abstract class and object cannot be created for it too
- Constructor can be created inside the Abstract Class for initializing the variables



```

4  class A(ABC): 1 usage
5      @abstractmethod
6      def m1(self): # abstract method
7          pass
8
9      @abstractmethod
10     def m2(self): # abstract method
11         pass
12     def m3(self): # Non-abstract method 1 usage
13         print("method 3 of Abstract class")
14
15 class B(A): 1 usage
16     def m1(self): 1 usage
17         print("m1 method implemented")
18     def m2(self): 1 usage
19         print("m2 method implemented")
20 obj = B() # we cant create object for abs-class but for child
21 obj.m1()
22 obj.m2()
23 obj.m3()

```

Module:

Modules in Python (Python Tutorial - Part 57)

- Modules in Python
 - Module is a python file containing a set of functions
 - Example: Calculator.py file containing functions say addition, subtraction, multiplication, division etc.
 - Accessing this module from a different py file by using the below statements:
 - import Calculator
 - Calculator.addition()
 - Calculator.subtraction()
 - from Calculator import addition, subtraction
 - addition()
 - subtraction()
 - from Calculator import *

- Finding the number of functions in a module using dir()
 - import Calculator
 - print(dir(Calculator))
- Instead of creating a set of functions inside the python file, create them under a Class or set of Classes
 - import Calculator
 - c1 = Calculator.Calc()
 - c1.addition()
 - c1.subtraction()

Demo.py	Calc.py
1 import Calc	1
2 from PythonPractice2.Calc import addition	2 def addition(a,b): 2 usages
3	3 sum = a+b
4 print(Calc.addition(a: 12, b: 12))	4 return sum
5	5
	6 def subtraction(a,b):
	7 sub = a-b
	8 return sub

from Calc import *	1
print(addition(a: 12, b: 12))	2 def addition(a,b): 1 usage
print(subtraction(a: 12, b: 5))	3 sum = a+b
	4 return sum
	5
	6 def subtraction(a,b): 1 usage
	7 sub = a-b
	8 return sub

from Calc import *	1 class A: 1 usage
from PythonPractice2 import Calc	2
print(dir(Calc)) # No of directories in Calc	3 def addition(self,a, b): 1 usage
obj = Calc.A()	4 sum = a + b
print(obj.addition(a: 12, b: 12))	5 return sum
print(obj.subtraction(a: 12, b: 5))	6
	7 def subtraction(self, a, b): 1 usage
	8 sub = a - b
	9 return sub

- Built-in Modules

- Built-in Modules in Python, have set of built-in functions
- Examples for Modules are **math** and **random**
- Importing Modules to use built-in functions
 - Example One:
 - `import math`
 - `print(math.sqrt(4))`
 - Example Two:
 - `import random`
 - `colors = ["orange", "red", "green"]`
 - `print(random.choice(colors))`

```
import math, random

colors = ["red", "orange", "green", "blue", "Black", "White", "Yellow", "Pink", "Brown", "Golden"]
print(random.choice(colors))
print(random.randint(a: 1, b: 100))
```

Packages:

Packages in Python (Python Tutorial - Part 58)

- Packages in Python
 - Packages are nothing but **folders**, created for organizing the stuff.
 - For using the functions of modules created inside different packages, we have to import the modules using the package names as below:
 - `import pack1.One`
 - `pack1.One.fun_one()`
 - `from pack1 import One, Two`
 - `One.fun_one()`
 - `Two.fun_two()`
 - `from pack1.One import *`
 - `fun_one()`



Exception Handling:

Exception Handling in Python (Python Tutorial - Part 59)

- Exception Handling in Python

- We get exceptions in python, due to invalid input to the program
- What will happen if the exceptions are not handled
 - print("Start")
 - a = 10/0
 - print(a)
 - print("End")
- Example: ZeroDivisionError, NameError etc.
- try:

- a = 10/0

- print(a)

- except ZeroDivisionError:

- print("Exception got handled")

- Exception Object

- except ZeroDivisionError as e:

- print("Exception got handled: ",e)

- What if the exception type provided in except block doesn't match

- Exception won't be handled

```
print("Start of the program")
try:
    a = 10/0
except ZeroDivisionError:
    print("Exception Handled")

print("Eng of the program")

print("Start of the program")
num = int(input())
try:
    a = 10/num
except ZeroDivisionError:
    print("Exception Handled")

print("Eng of the program")
```

```

print("Start of the program")
a = input()
try:
    a = 10+a
except TypeError:
    print("Exception Handled")

print("Eng of the program")

```

```

print("Start of the program")

try:
    print(a)
except NameError as n:
    print("Exception Handled", n)

print("Eng of the program")

```

- except ZeroDivisionError:
 - print("Exception got handled")
- Exception Object
 - except ZeroDivisionError as e:
 - print("Exception got handled: ",e)
- What if the exception type provided in except block doesn't match
 - Exception won't be handled
 - Provide NameError in place of above ZeroDivisionError
- Multiple except blocks are possible
- else: block gets executed when exception is not thrown
- finally: block gets executed no matter what
- Raising exception using raise keyword
 - raise ValueError("Age should be less than 10")

```

print("Start of the program")
#a =10
try:
    print(a)
except NameError as n:
    print("Exception Handled", n)
else:
    print("inside else block")
finally: # finally will be printed in each condition
    print("inside finally block")
print("Eng of the program")

```

```
print("Start of the program")
raise ZeroDivisionError
print("Eng of the program")
```

```
print("Start of the program")
age = int(input())
try:
    if age<5:
        raise TypeError("below 5 years is not allowed")
    else:
        print(" You are eligible")
except TypeError as n:
    print("Exception got handled")

print("Eng of the program")
```

Lambda Function:

Lambda function in Python (Python Tutorial - Part 60)

- Lambda function in Python
 - A function which has no name, and can intake any number of arguments and one expression is Lambda function
 - Examples:
 - `sum = lambda a, b: a+b`
 - `print(sum(5,4))`
 - lambda function reduces the number of lines of code to be written in a normal function

```
sum = lambda a,b : a+b
print(sum( a: 12, b: 12))
```

*Args:

Using *args in Python (Python Tutorial - Part 61)

- Using *args in Python
 - Using *args in function parameter
 - Here args can be named as anything
 - `def fun_one(*args):`
 - `for i in args:`
 - `print(i)`
 - `fun_one(9,1,5,7)`
 - Using *args as arguments in function calling
 - `def fun_one(a,b,c,d):`
 - `print(a,b,c)`
 - `args = [9,1,5,7]`
 - `fun_one(*args)`

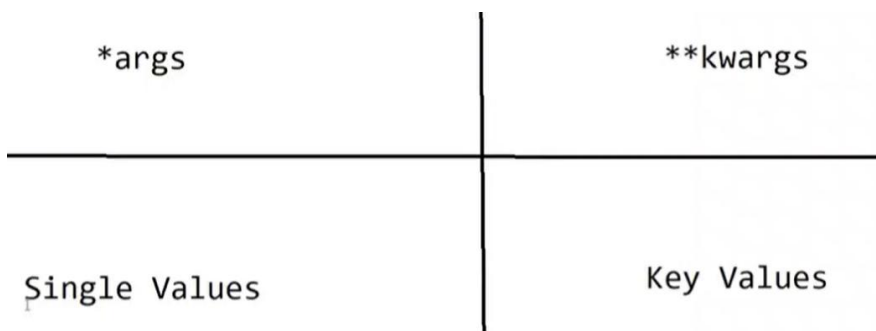
```
def sample(*args): 2 usages
    for i in args:
        print(i)
sample(*args: 9,5,2,6)
```

```
def sample1(*v):
    for i in v:
        print(i)
sample(*args: 9,5,2,6)
```

```
def sample(a,b,c): 1 usage
    print(a,b,c)
args = [1,2,3]
sample(*args) # Arguments
```

Using ******kwargs in Python (Python Tutorial - Part 62)

- Using **kwargs in Python
 - Example One
 - def fun_one(Name,Experience,Location):
 - print(Name,Experience,Location)
 - d = {"Name":"Arun","Experience":12,"Location":"Hyderabad"}
 - fun_one(**d)
 - Output: Arun 12 Hyderabad
 - Example Two
 - def fun_one(**kwargs):
 - for i,j in kwargs.items():
 - print(i,j)
 - fun_one(name="Arun",experience=12,location="Hyderabad")



```
def sample(**kwargs): 1 usage
    for k,v in kwargs.items():
        print(k,v)
sample(name="Popal",lastName="Furqani",Location="VA")
```

```
def sample(name,lastName,location): 1 usage
    print(name,lastName,location)
d = {"name":"Popal","lastName":"Furgani","location":"VA"} # dictionary
sample(**d)
```

Unpacking Collections or range():

Unpacking Collections or range() into individual variables (Python Tutorial - Part 63)

Skip countdown

- Unpacking List into individual variables
- Unpacking Tuple into individual variables
- Unpacking Set into individual variables
- Unpacking Dictionary into individual variables
- Unpacking range(num) into individual variables
- Practical Demonstration

```
a,b,c,d = [9,2,5,6] # unpack list
print(a)
print(b)
print(c)
print(d)
s = {"name":"Khan","Location":"VA","Phone":"7573828282"} # unpack dictionary
e,f,g = s.values()
print(e)
print(f)
print(g)
```

Complex Data Type in Python (Python Tutorial - Part 64)

- Numerical Data Types in Python - int, float and complex
- We generally don't use this data type in real time, but can learn it for knowledge purpose
- Complex data type is used while developing scientific applications where complex mathematical operations are required etc.
- Hence we don't have to learn in depth
- Syntax for Complex Data Type:
 - real + imag j
 - Example: 3 + 5 j
 - Here j is square root of -1
- Printing a complex data type variable
- Printing the type of a complex data type variable using type(variable)
- Printing the real value of complex data type variable
- Printing the imaginary value of complex data type variable
- Performing operations like addition, multiplication etc. on complex data type variables
- Real value of complex data type variable is optional

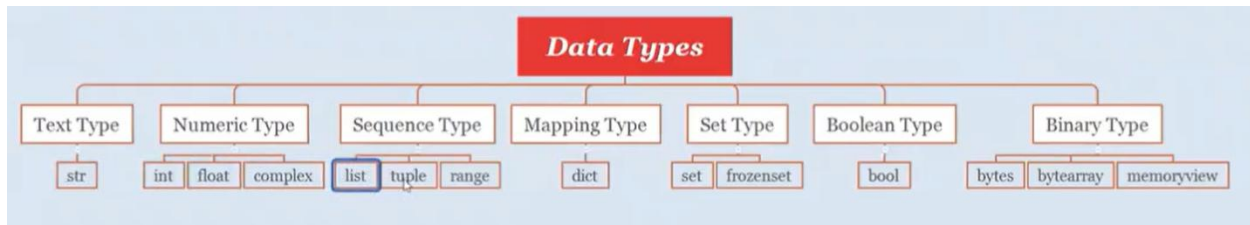
range Data Type in Python (Python Tutorial - Part 65)

- range is one of the Data Types in Python
- Example: a = range(3)
- Printing the range data type variable
- Printing the data type of the above variable using type() function
- Using for loop for printing the range values
- Different varieties of using range() function
 - range(number)
 - range(begin,end)
 - range(begin,end,increment/decrement)
- Using index values for accessing the range values
 - range(a[0])
 - Slicing - range(a[2:5])
- Practical Demonstration

```
r = range(4) # 0,1,2,3,4
for v in r:
    print(v)
r = range(1,5) # 1,2,3,4
for v in r:
    print(v)
r = range(1,10,2) # 1,3,5,7,9
for v in r:
    print(v)
r = range(10,0,-1) # 9,8,7,6,5,4,3,2,1
for v in r:
    print(v)
```

Complete List of Data Types in Python (Python Tutorial - Part 66)

- The following is the complete list of data types in Python:
 - Text Type - str
 - Numeric Type - int, float and complex
 - Sequence Type - list, tuple and range
 - Mapping Type - dict
 - Set Type - set and frozenset
 - Boolean Type - bool
 - Binary Type - bytes, bytearray and memoryview



```
s = {4,5,7,8,3} # Set
s.add(6)
print(s)
fs = frozenset({4,5,7,8,3}) # Frozen Set is immutable
fs.add(6)
print(fs)
```

Escape Characters in Python (Python Tutorial - Part 68)

- Escape Characters are used in String text for inserting illegal characters, tab space, new line and many more
- The below are major Escape Characters we can learn:
 - \'
 - \"
 - \\
 - \n
 - \r
 - \t
 - \b
- Practical Demonstration

Using `bool()` function in python for evaluating values as True or False (Python Tutorial - Part 69)

- Practical Demonstration of the below:
 - `bool(0)` returns False
 - `bool(any number)` returns True
 - `bool("")` returns False
 - `bool("Sample String")` return True
 - `bool(Empty Collections)` return False
 - `bool(Collections containing values)` return True
 - `bool(None)` returns False
 - `bool(False)` returns False
 - `bool(True)` returns True

Short Hand if and if .. else Statements in Python (Python Tutorial - Part 70)

- Short Hand if statement
 - if `a > b`: `print("a is greater than b")`
- Short Hand if .. else statement
 - `print("a is greater than b")` if `a > b` else `print("b is greater than a")`
- Practical Demonstration

```
a = 5
b = 4
# if a > b:
#     print("a is greater than b")
# else:
#     print("b is greater than a")
print("a is greater than b") if a > b else print("b is greater than a")
```

User `iter()` function in Python with List, Tuple, Set and Dictionary (Python Tutorial - Part 71)

- Using `iter()` with List
- Using `iter()` with Tuple
- Using `iter()` with Set
- Using `iter()` with Dictionary
- Practical Demonstration

```

a = [4,2,6,7,9] # list with iter
iterate = iter(a)

print(next(iterate))
print(next(iterate))
print(next(iterate))

```

Date and Time in Python (Python Tutorial - Part 72)

- import `datetime` module
- `print(datetime.datetime(1984,11,25,23,30,27))`
- `d = datetime.datetime(1984,11,25,23,30,27)`
- `print(d)`
- `print(d.date())`
- `print(d.year)`
- `print(d.month)`
- `print(d.day)`
- `print(d.hour)`
- `print(d.minute)`
- `print(d.second)`
- `print(d.strftime("%dth %B,%Y"))`
- `print(datetime.datetime.now())`
- Practical Demonstration

Directive	Description	Example	Try it
%a	Weekday, short version	Wed	Try it »
%A	Weekday, full version	Wednesday	Try it »
%w	Weekday as a number 0-6, 0 is Sunday	3	Try it »
%d	Day of month 01-31	31	Try it »
%b	Month name, short version	Dec	Try it »
%B	Month name, full version	December	Try it »
%m	Month as a number 01-12	12	Try it »
%y	Year, short version, without century	18	Try it »
%Y	Year, full version	2018	Try it »
%H	Hour 00-23	17	Try it »
%I	Hour 00-12	05	Try it »
%p	AM/PM	PM	Try it »
%M	Minute 00-59	41	Try it »

```
import datetime
dt = datetime.datetime(year: 1988, month: 2, day: 12, hour: 23, minute: 40, second: 22)
dt1 = datetime.datetime.now() # will print update date and time
print(dt1)
print(dt)
print(dt.date())
print(dt.hour)
print(dt.strftime("%dth %B,%Y")) # 12th February, 1988
```

Regular Expressions in Python (Python Tutorial - Part 73)

- Regular Expressions (RegEx) are used to check whether the search pattern is available in the string
 - Pattern: Arun
 - String: My name is Arun Motoori
- import re
- re.search(pattern,text)
- Different Patterns
 - python
 - Only matches with python text, but won't match with Python
 - [Pp]ython
 - Matches with either Python or python
 - ye[sp]
 - Matches with either yes or yep
 - [sfk]it
 - Matches with sit or fit or kit
- [0-9]am
 - Matches with 9am or 5am, but won't match with sam
- [a-z]et
 - Matches with set or let, but won't match with 9et
- [A-Z]et
 - Matches with Set or Let, but won't match set or let or 9et
- [a-zA-Z0-9]
 - Matches with Set, set and 9et
- [^0-9]et
 - Matches with set or let, but not with 9et
- se[a-z]
 - Matches with sez, sem, set etc, but not with seZ or se9
- s[[^]aeiou]t

- \d
 - matches a digit and is equal to specifying [0-9]
 - Example: abc\defg
 - Accepts abc9efg and Rejects abcdefg
- \D
 - matches a non-digit and is equal to specifying [^0-9]
 - Example: abc\Defg
 - Accepts abcdefg and Rejects abc9efg
- \w
 - Matches a single word character and is equal to specifying [A-Za-z0-9_]
- \W
 - Matches a single non-word character and is equal to specifying [^A-Za-z0-9_]
- ^My
 - Starts with My
- Arun\$
 - Ends with Arun
- \w
 - Matches a single word character and is equal to specifying [A-Za-z0-9_]
- \W
 - Matches a single non-word character and is equal to specifying [^A-Za-z0-9_]
- ^My
 - Starts with My
- Arun\$
 - Ends with Arun
- A..n
 - . matches any character except newline
- ^My.*Arun\$
 - Starts with My and Ends with Arun
 - * repeats the . expression 0 or any number of times in this example
 - MyArun is accepted
 - My name is Arun is accepted

`^My.*Arun$`

- Starts with My and Ends with Arun
- * repeats the . expression 0 or any number of times in this example
- MyArun is accepted
- My name is Arun is accepted

`^My.+Arun$`

- Starts with My and Ends with Arun
- + repeats the . expression 1 or any number of times in this example
- MyArun is not accepted

`^My.?Arun$`

- Starts with My and Ends with Arun
- ? repeats the . expression 0 or 1 number of times in this example
- MyArun and My Arun are accepted, My name is Arun is not accepted
- My name is Arun is not accepted

`^My.{2}Arun$`

- Starts with My and Ends with Arun
- {2} repeats the . expression exactly two times in this example

```
import re # search inbuilt function is used for search

pattern = "[POPALpopal]"
print("Enter you input here")
text = input()
if re.search(pattern,text): # search pattern in text.
    print("Pattern got matched")
else: print("pattern didn't match")
```

```
import re # Import the regex module

pattern = r"\wopal" # Raw string notation is recommended
print("Enter your input here:")
text = input()

if re.search(pattern, text): # Search pattern in text.
    print("Pattern matched!")
else:
    print("Pattern didn't match.")
```

Will accept Popal popal 4opal any thing

Table of Contents

Python.....	1
Key Features of Python:	1
Common Use Cases:	1
Download, and Install	1
Print Statement	2
Variables	3
Data Types in Python	4
Operators.....	5
Comments.....	5
String Concatenation in Print Statement	6
Selection/Decision Control Statements	8
Repetition Loop:.....	9
Functions:	11
Built-in Functions	15
Local & Global Variables:	17
Collections:	19
List:.....	19
Tuple:	21
Set:	23
Dictionary:	25
Difference between collection:.....	28
Strings:	28
In & not in Operators:	30
File Handling:	31
Class and Object:	34
Self keyword:	35
Static and Instance Variable and Methods:	38
Inheritance:	40
Type of Inheritance:	41

Override Method, Variables:	44
Overloading Methods, Variables:	45
Getter and Setter	47
Encapsulation:.....	48
Abstraction:.....	49
Module:.....	50
Packages:.....	52
Exception Handling:	53
Lambda Function:	55
*Args:	55
Unpacking Collections or range():.....	57